***BRACT's***

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:C Batch: 3 Practical No: 1

Roll no: 69 PRN No: 12415121 Name of Student: Kunal Patil

**Subject Name : Deep Learning**

## Problem Statement : **Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.**

## Algorithm :

Based on your **Fashion MNIST TensorFlow/Keras** code, here is the algorithm you can write in your assignment.

### Algorithm: Image Classification using TensorFlow/Keras (Fashion MNIST)

1. Start the program.
2. Import the required libraries:
    - TensorFlow/Keras
    - Matplotlib
    - Scikit-learn modules (if required)
3. Load the **Fashion MNIST** dataset into training and testing sets.
4. Normalize the pixel values by dividing each pixel by **255.0** to scale them between **0 and 1**.
5. Display the shape of the training and testing datasets.
6. Visualize a sample image from the training dataset.

7. Create a Sequential Neural Network model.
8. Add a **Flatten** layer to convert the 28 × 28 image into a one-dimensional vector.
9. Add a **Dense hidden layer** with **128 neurons** and **ReLU** activation.
10. Add an **output layer** with **10 neurons** and **Softmax** activation for classifying the 10 fashion categories.
11. Compile the model using:
    - Optimizer: **Adam**
    - Loss Function: **Sparse Categorical Crossentropy**
    - Metric: **Accuracy**
12. Train the model using the training dataset for **7 epochs**.
13. Evaluate the trained model using the testing dataset.
14. Display the test accuracy.
15. Save the trained model in **.keras** format.
16. Stop the program.

## Code :

```
#Importing libraries
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

#Loading and spliting dataset
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.fashion_mnist.load_data()

#Scaling dataset
train_images = train_images /255.0
test_images = test_images / 255.0
```

```
#Visualizing data
print(train_images.shape)
print(test_images.shape)
print(train_labels)

plt.imshow(train_images[1], cmap='gray')
plt.show()

#Defining neural network models
#Input layer
my_model = tf.keras.models.Sequential()
#Hidden layer
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

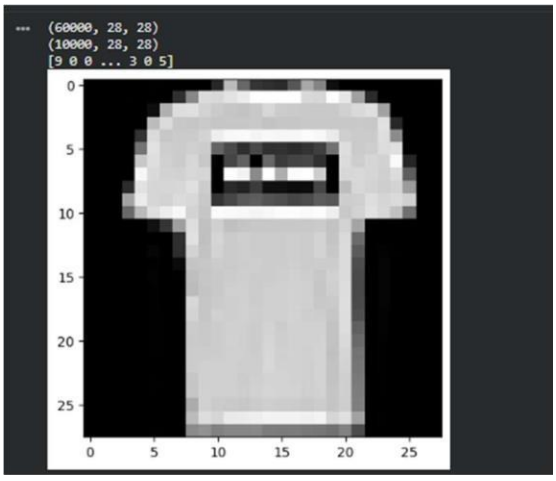
#Compiling model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

#Training Model
my_model.fit(train_images, train_labels, epochs=7)

#Testing Accuracy
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

#Saving Model
my_model.save('my_fashion_mnist_model.keras')
```

## Output :




```
Epoch 1/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.6857 - loss: 3.3585
Epoch 2/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.7794 - loss: 0.6368
Epoch 3/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.8076 - loss: 0.5603
Epoch 4/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8191 - loss: 0.5322
Epoch 5/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8240 - loss: 0.5116
Epoch 6/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8270 - loss: 0.5014
Epoch 7/7
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8333 - loss: 0.4918
<keras.src.callbacks.history.History at 0x79636a1ce000>

val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8202 - loss: 0.5426
Test accuracy:  0.8202000260353088
```